

Padrões de Projeto: Singleton

Jhony Luiz de Almeida

O problema

- Garantir que um recurso específico possua uma única instância, compartilhada por toda a aplicação, não permitindo que sejam criadas novas instâncias acidentalmente.
- Exemplos:
 - Conexão única com um recurso como base de dados
 - Objetos sem estado que podem ser compartilhados de forma segura entre várias threads (stateless)
 - Configurações globais da aplicação

A solução

- A implementação de um singleton na prática é simples, consiste basicamente em restringir o acesso ao construtor de uma classe apenas a ela própria, e fornecer um método estático que inicialize e retorne sempre a mesma instância
- Sem construtores públicos, é garantido que nenhuma outra classe poderá gerar novas instâncias da classe Singleton
- O método estático em algumas implementações pode inicializar a instância apenas na primeira vez que é chamado, apenas retornando-a em chamadas posteriores

Padrões de Projeto: Strategy

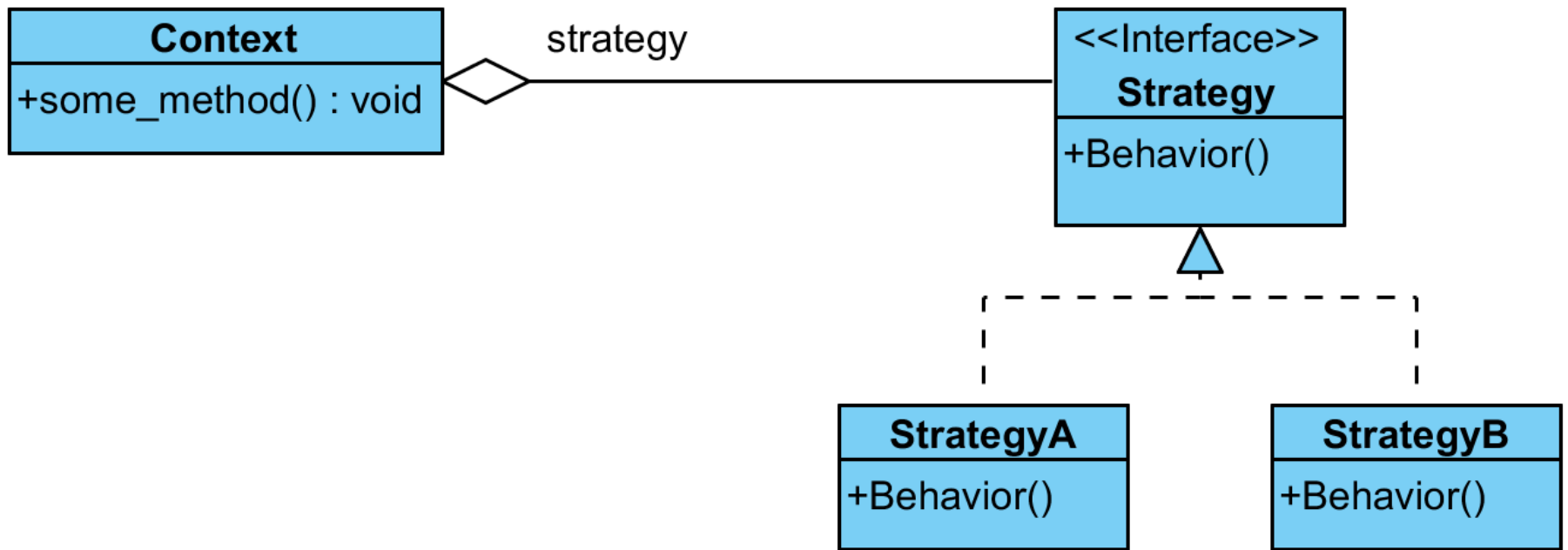
Jhony Luiz de Almeida

O problema

- Um algoritmo que executa certo processo possui várias regras diferenciadas para N cenários
- Exemplos:
 - Calcular custos de frete baseado na modalidade escolhida
 - Gerar conteúdo de um relatório baseado no formato do arquivo
 - Traçar rotas entre pontos baseado no meio de transporte usado
- Uma abordagem em código procedural seria criar diversas condicionais em sequência checando cada uma das regras, tornando o código cada vez mais complexo de manter, estender e debugar

A solução

- O padrão *Strategy* propõe que a parte do algoritmo que possui regras distintas seja abstraída por uma interface, denominada estratégia, e cada variação seja uma classe concreta que a implementa
- A classe responsável por dar início à execução do algoritmo passa a ser o Contexto, que guarda a referência da estratégia utilizada bem como os dados necessários para executá-la



Padrões de Projeto: Facade

Jhony Luiz de Almeida

O problema

- Um sistema ou módulo existente (legado) precisa ser usado para executar certas ações, mas demanda várias sub-operações de baixo nível para isso
- Exemplo: um antigo sistema de compras que trabalha com carrinhos de compra, onde para poder executar uma simples compra, deve ser criado um novo carrinho, adicionados os produtos, invocada a função de checkout para calcular o valor a ser pago, adicionado um método de pagamento, etc...
- Se temos vários clientes que usam este sistema, todos precisam lidar com todo este processo complexo para efetuar compras

A solução

- No padrão *Facade* uma classe de “fachada” é criada para esconder a complexidade de um subsistema, expondo métodos que abstraem operações complexas e podem ser acessados por múltiplos clientes
- Um facade de compras por exemplo, poderia oferecer um método *efetuarCompra* que já recebe a lista de produtos e o meio de pagamento, e realiza todas as operações necessárias para efetivar a compra nos subsistemas

Simplificação com Facade

